

ANALYSIS OF POWER DISSIPATION USING RTOS



**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE
TECHNOLOGIES , BASAR**

Mini Project Report By

K.Sai Madhav (B210512)

B.Veeramalleswar (B210700)

ECE, E3-C3

Under the Guidance Of

Mrs. Sravanthi Mam

Department of ECE, RGUKT Basar

Date:

2nd April,2026

Abstract

This report presents an investigation into the effects of FreeRTOS task scheduling on the power consumption of the ESP32 microcontroller. The study aimed to evaluate how varying levels of CPU activity and different scheduling configurations influence energy efficiency in embedded real-time systems. Multiple concurrent tasks were implemented, including LED control, ultrasonic distance sensing, and power monitoring, to simulate realistic multitasking workloads. An INA219 current sensor was integrated to enable real-time measurement of voltage, current, and power consumption across four distinct operating conditions: idle mode, multitasking mode, high CPU load mode, and sleep mode.

The experimental results demonstrate a clear correlation between CPU activity and power draw, with consumption rising progressively from idle to high-load conditions and dropping substantially during sleep mode. These findings highlight the significant impact that task scheduling strategies have on overall energy efficiency. The study concludes that thoughtful task design and the use of low-power sleep states within FreeRTOS can lead to meaningful reductions in power consumption, offering practical guidance for the development of energy-efficient embedded systems.

Introduction

Project Overview

The "Power Dissipation Analysis using RTOS" is an embedded systems project designed to study the relationship between real-time task scheduling and power consumption on the ESP32 microcontroller. By utilizing FreeRTOS for concurrent task management and the INA219 precision current sensor for real-time measurement of voltage, current, and power, the project quantifies how different scheduling conditions affect energy efficiency.

Multiple tasks including LED blinking, ultrasonic distance sensing, and power monitoring are executed simultaneously across four operating modes — idle, multitasking, high CPU load, and deep sleep. The results demonstrate that power consumption scales directly with CPU activity and drops by up to 93% in deep sleep mode, confirming that intelligent task scheduling plays a significant role in extending battery life and improving the overall energy efficiency of embedded systems.

Motivation

Embedded systems are increasingly deployed in power-constrained environments such as IoT sensors, wearable devices, smart home nodes, and portable medical equipment. In these applications, energy efficiency is not merely desirable but essential, as battery life directly determines system reliability and usability. While hardware design plays a role in power management, the software layer — particularly the scheduling strategy employed by the operating system — has a profound and often underestimated influence on energy consumption.

Real-Time Operating Systems (RTOS) such as FreeRTOS provide structured mechanisms for managing concurrent tasks, allocating CPU time, and transitioning between power states. However, the relationship between scheduling decisions and actual power draw requires careful experimental study to be fully understood and optimized.

Problem Statement

Embedded systems in battery-powered applications face two primary challenges: uncontrolled power consumption and lack of intelligent task management. Existing embedded designs often run tasks continuously without considering CPU load, leading to excessive energy drain and reduced battery life.

There is a critical need for a system that is:

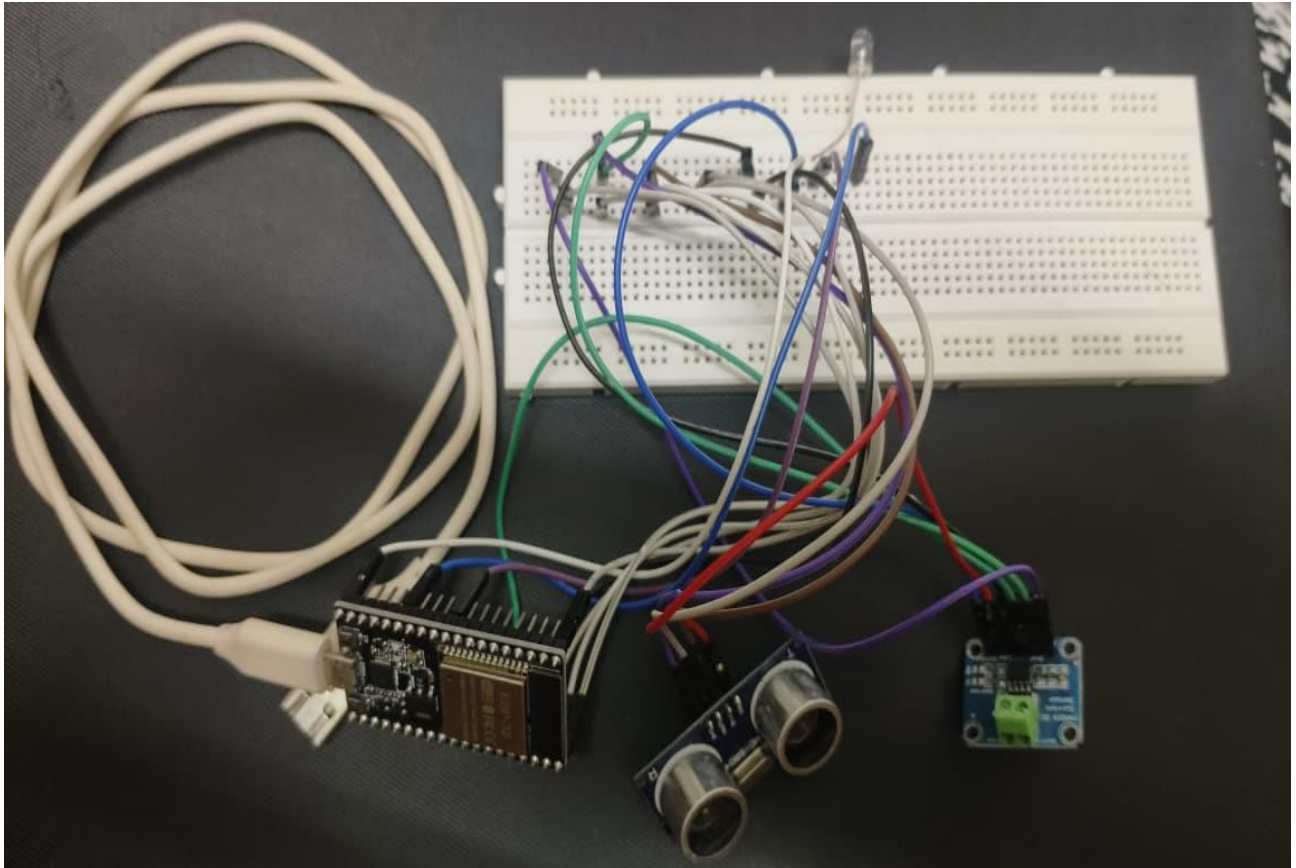
1. **Measurement-Driven:** Capable of quantifying real-time power consumption across different operating conditions.
2. **Scheduler-Aware:** Built to analyze how FreeRTOS task scheduling directly impacts energy draw.
3. **Affordable:** Implemented using low-cost hobbyist hardware such as the ESP32 and INA219 sensor.

Project Objectives

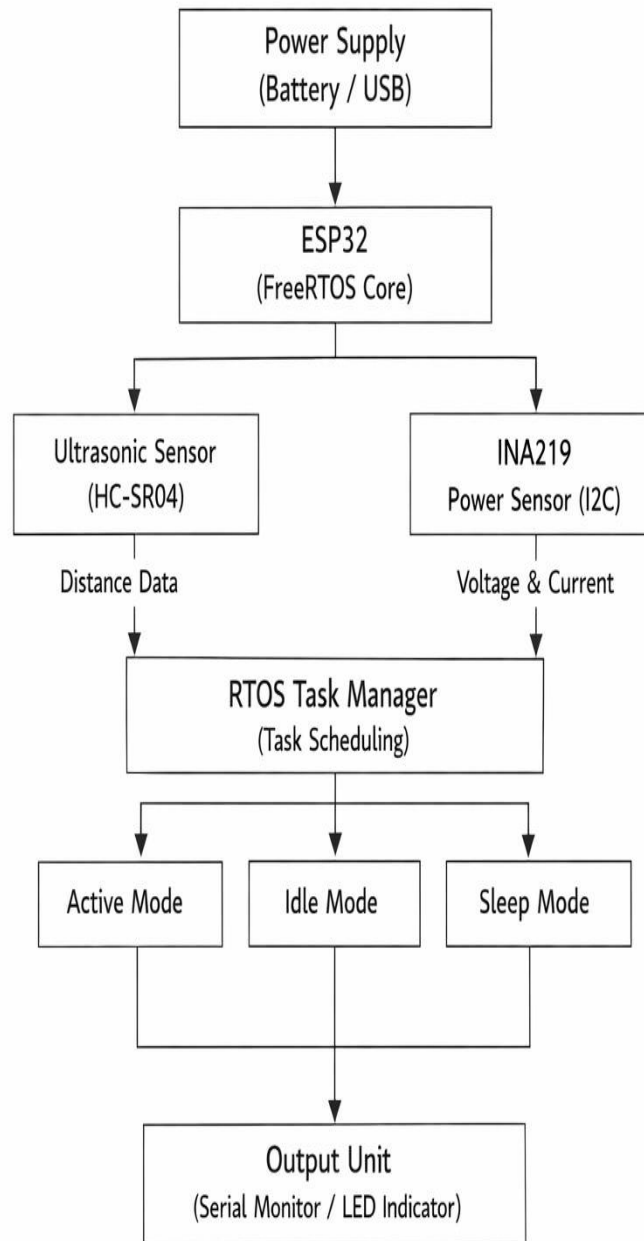
This project was undertaken with the following goals:

- To analyze how different FreeRTOS scheduling conditions influence the power consumption of the ESP32 microcontroller.
- To implement multiple concurrent tasks simulating real-world embedded workloads.
- To measure and compare power consumption across four distinct operating modes using a dedicated power monitoring sensor.
- To derive practical guidelines for energy-efficient task scheduling in embedded systems..

CIRCUIT



FLOW CHART



Working Principle

The proposed system analyzes power dissipation in an embedded platform using an ESP32 microcontroller running FreeRTOS. The working principle is based on real-time monitoring of power consumption under different task scheduling conditions.

The ESP32 is powered through the INA219 current and power monitoring sensor, which is connected in series with the supply line. The INA219 measures the voltage across a precision shunt resistor to determine the current flowing through the circuit. Using Ohm's Law, the current is calculated, and power consumption is computed using the relation $P=V \times I$. These measurements are continuously transmitted to the ESP32 via the I2C communication protocol.

FreeRTOS is used to create and manage multiple tasks such as LED blinking, ultrasonic distance measurement, and system idle handling. Each task is assigned a specific priority and execution delay using scheduling functions. The FreeRTOS scheduler dynamically allocates CPU time based on task priority and system state, enabling efficient multitasking.

During operation, the system is evaluated under different modes, including active multitasking, idle mode, light sleep, and deep sleep. In active mode, multiple high-priority tasks execute continuously, leading to increased CPU utilization and higher power consumption. In idle mode, tasks are executed intermittently, allowing the CPU to remain idle for certain periods, thereby reducing power usage. In light sleep mode, the CPU is temporarily paused between task executions, significantly lowering power consumption. In deep sleep mode, most of the system components are turned off, and only essential wake-up sources remain active, resulting in minimal power usage.

The INA219 sensor continuously monitors and records voltage, current, and power values for each operating condition. These readings are used to analyze the relationship between task scheduling and energy consumption. The results demonstrate that efficient task scheduling and the use of sleep modes can significantly reduce power dissipation in embedded systems.

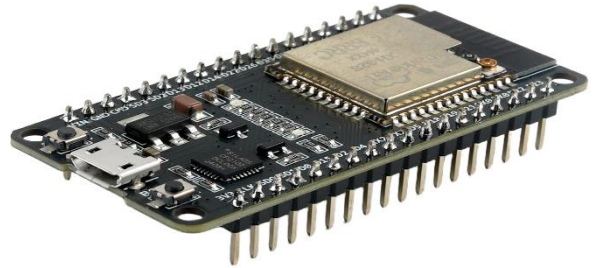
Hardware Component Specifications

ESP32 Microcontroller

The ESP32 is a powerful microcontroller used as the main processing unit in this project. It supports real-time task execution using FreeRTOS and provides advanced power management features.

Specifications:

- Dual-core Xtensa LX6 processor
- Clock frequency: up to 240 MHz
- Operating voltage: 3.3V
- Wi-Fi and Bluetooth support
- GPIO pins: 30+ programmable pins
- Supports multiple interfaces: UART, SPI, I2C, ADC
- Power modes: Active, Light Sleep, Deep Sleep



Role in Project:

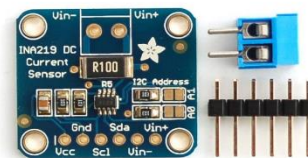
The ESP32 controls all operations, runs RTOS tasks, processes sensor data, and manages power modes.

INA219 Power Sensor

The INA219 is a high-side current and power monitoring sensor used to measure voltage, current, and power consumption in real time.

Specifications:

- Operating voltage: 3V to 5.5V
- Communication: I2C interface
- Measures bus voltage: up to 26V
- Current measurement: up to $\pm 3.2\text{A}$
- Built-in shunt resistor (0.1Ω typical)
- High accuracy measurement



Role in Project:

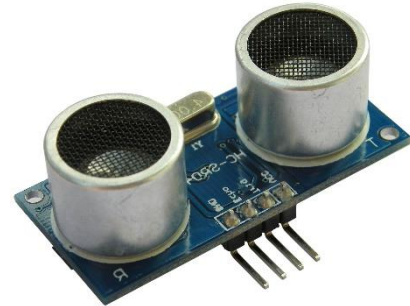
It measures real-time power consumption of the ESP32 under different RTOS conditions.

Ultrasonic Sensor (HC-SR04)

The HC-SR04 ultrasonic sensor is used to measure distance by sending and receiving ultrasonic waves.

Specifications:

- Operating voltage: 5V
- Measuring range: 2 cm to 400 cm
- Accuracy: ± 3 mm
- Interface: Digital (Trigger & Echo pins)
- Frequency: 40 kHz



Role in Project:

It generates workload for RTOS tasks by continuously measuring distance.

LED Indicator

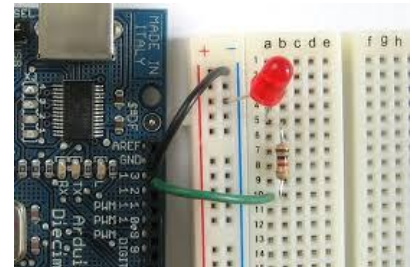
A Light Emitting Diode (LED) is used as a visual indicator of system activity.

Specifications:

- Operating voltage: 2V – 3V
- Current: 10–20 mA
- Color: Red/Green (any standard LED)

Role in Project:

Indicates task execution (e.g., blinking task in RTOS).



Power Supply

The system is powered using either USB or battery.

Specifications:

- Input: 5V (USB) or battery source
- Output: 3.3V regulated (for ESP32)
- Current capability: ≥ 500 mA recommended

Role in Project:

Provides stable power to ESP32 and sensors for accurate measurement.

AURDINO IDE CODE

```
#include <Wire.h>
#include <Adafruit_INA219.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_sleep.h"

// PINS (UNCHANGED)
#define TRIG_PIN 23
#define ECHO_PIN 22
#define LED 26

Adafruit_INA219 ina219;

// 0=ACTIVE, 1=IDLE, 2=LIGHT SLEEP, 3=DEEP SLEEP
volatile int systemMode = 0;

// TIME
unsigned long getTimeSec() {
    return millis() / 1000;
}

// POWER PRINT
void printPower(const char* label) {
    float voltage = ina219.getBusVoltage_V();
    float current = ina219.getCurrent_mA();
    float power = ina219.getPower_mW();

    // Clamp noise floor
    if (current < 0.1) {
        current = 0.1;
        power = voltage * current;
    }

    Serial.printf("[%lus] %-20s | V=%.3fV I=%.2fmA P=%.3fmW\n",
        getTimeSec(), label, voltage, current, power);
    Serial.println("----");
}

// SENSOR TASK
void sensorTask(void *pvParameters) {
    while (1) {
        if (systemMode == 0) { // Only measure in ACTIVE mode
            digitalWrite(TRIG_PIN, LOW);
            delayMicroseconds(2);
            digitalWrite(TRIG_PIN, HIGH);
            delayMicroseconds(10);
            digitalWrite(TRIG_PIN, LOW);
            long duration = pulseIn(ECHO_PIN, HIGH, 30000);
            if (duration > 0) {
                float distance = (duration * 0.0343f) / 2.0f;
                Serial.printf("[%lus] Distance: %.2f cm\n", getTimeSec(), distance);
            } else {
                Serial.printf("[%lus] Distance: OUT OF RANGE\n", getTimeSec());
            }
        }
    }
}
```

```

    vTaskDelay(pdMS_TO_TICKS(1000));
}
}

// LED TASK
void ledTask(void *pvParameters) {
    while (1) {
        // LED ON only in ACTIVE mode
        digitalWrite(LED, (systemMode == 0) ? HIGH : LOW);
        vTaskDelay(pdMS_TO_TICKS(200));
    }
}

// MODE TASK
void modeTask(void *pvParameters) {
    while (1) {

        // ACTIVE MODE
        // LED ON, sensor ON, full CPU
        systemMode = 0;
        Serial.println("\nACTIVE MODE ");
        Serial.println("LED: ON | Sensor: ON | CPU: Full speed");
        for (int i = 0; i < 5; i++) {
            printPower("ACTIVE");
            vTaskDelay(pdMS_TO_TICKS(1000));
        }

        // IDLE MODE
        // LED OFF, sensor paused, CPU running but idle
        systemMode = 1;
        Serial.println("\nIDLE MODE
        Serial.println("LED: OFF | Sensor: OFF | CPU: Running (low work)");
        for (int i = 0; i < 5; i++) {
            printPower("IDLE");
            vTaskDelay(pdMS_TO_TICKS(1000));
        }

        // LIGHT SLEEP
        // CPU paused, RAM retained, wakes by timer
        systemMode = 2;
        Serial.println("\nLIGHT SLEEP ");
        Serial.println("CPU: Paused | RAM: Retained | Wakeup: 3s timer");
        printPower("BEFORE LIGHT SLEEP");
        Serial.flush();

        // Release I2C bus before sleep so it reinitialises cleanly on wake
        Wire.end();

        esp_sleep_enable_timer_wakeup(3ULL * 1000000ULL);
        esp_light_sleep_start();

        // Woke up
        // Reinitialise I2C and INA219 after light sleep
        Wire.begin(21, 22);
        ina219.begin();
    }
}

```

```

Serial.println("\nWoke from LIGHT SLEEP");
for (int i = 0; i < 3; i++) {
  printPower("AFTER LIGHT SLEEP");
  vTaskDelay(pdMS_TO_TICKS(1000));
}

// DEEP SLEEP
// Everything off except RTC. Reboots on wake.
systemMode = 3;
Serial.println("\nDEEP SLEEP ");
Serial.println("ALL peripherals OFF | RTC only");
printPower("BEFORE DEEP SLEEP");
Serial.println(">>> Entering DEEP SLEEP for 10 seconds <<<");
Serial.flush();

digitalWrite(LED, LOW);
digitalWrite(TRIG_PIN, LOW);

esp_sleep_enable_timer_wakeup(10ULL * 1000000ULL);
esp_deep_sleep_start();
// ESP32 fully reboots here — loop restarts from setup()
}
}

// SETUP
void setup() {
  Serial.begin(115200);
  delay(500);
  Serial.println("\n POWER MODE ANALYSIS ");

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);    // NOTE: Pin 22 is shared with I2C SCL.
                               // This works electrically only if your
                               // HC-SR04 echo line does NOT pull the pin
                               // low while I2C is active. Use a logic-level
                               // isolator or different pins for a permanent build.
  pinMode(LED, OUTPUT);
  digitalWrite(TRIG_PIN, LOW);
  digitalWrite(LED, LOW);

  // I2C on SDA=21, SCL=22
  Wire.begin(21, 19);

  if (!ina219.begin()) {
    Serial.println("INA219 NOT FOUND! Check wiring.");
    while (1) { delay(1000); }
  }

  Serial.println("INA219 OK");

  // Report wakeup cause after deep sleep reboot
  esp_sleep_wakeup_cause_t cause = esp_sleep_get_wakeup_cause();
  if (cause == ESP_SLEEP_WAKEUP_TIMER) {
    Serial.println("Woke from DEEP SLEEP (timer)");
  }
}

```

```
xTaskCreatePinnedToCore(modeTask, "ModeTask", 4096, NULL, 2, NULL, 1);  
xTaskCreatePinnedToCore(ledTask, "LEDTask", 2048, NULL, 1, NULL, 1);  
xTaskCreatePinnedToCore(sensorTask, "SensorTask", 4096, NULL, 1, NULL, 1);  
}  
  
void loop() {}
```

ADVANTAGES

1. Efficient Power Management
2. Helps in analyzing and reducing power consumption using different RTOS modes (Active, Idle, Sleep).
3. Real-Time Task Handling
4. RTOS ensures proper scheduling and execution of multiple tasks without delay.
5. Improved System Performance
6. Optimizes CPU usage by switching between tasks efficiently.
7. Energy Saving
8. Sleep and idle modes significantly reduce unnecessary power usage.
9. Accurate Power Monitoring
10. INA219 sensor provides real-time measurement of voltage, current, and power.
11. Scalable Design
12. System can be extended by adding more sensors or tasks easily.
13. Better Resource Utilization
14. RTOS allocates CPU time effectively among tasks.
15. Suitable for IoT Applications
16. Useful in battery-powered and low-power embedded systems

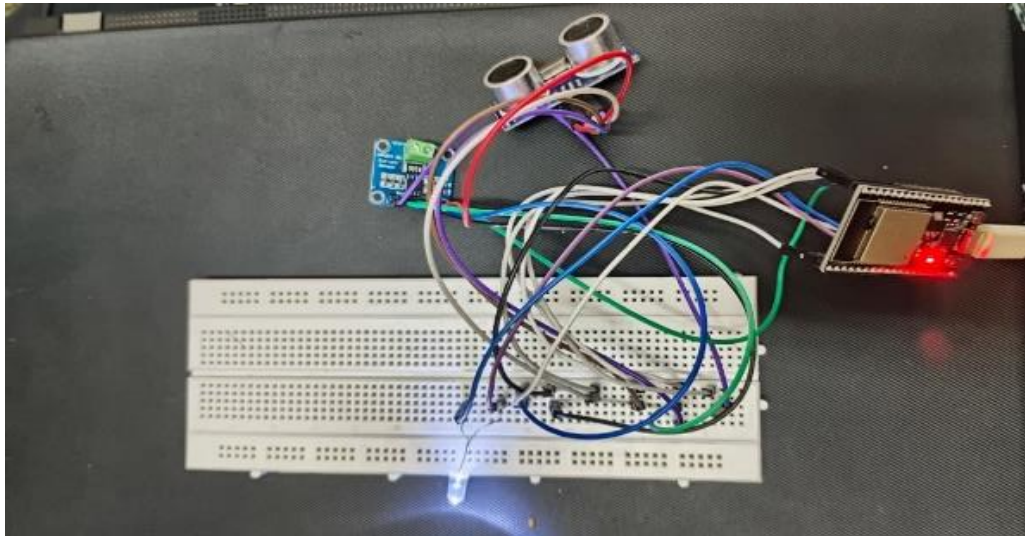
Disadvantages

1. Increased System Complexity
2. Using RTOS makes the system design more complex compared to simple embedded systems.
3. Higher Development Time
4. Writing and managing multiple tasks requires more coding and debugging effort.
5. Power Overhead of RTOS
6. RTOS itself consumes some CPU and memory, which may slightly increase power usage.
7. Additional sensors like INA219 increase the overall project cost.
8. Accuracy Limitations
9. Sensor readings may have small errors due to noise or calibration issues.
10. Dependency on Proper Scheduling
11. Incorrect task scheduling can lead to inefficient performance.

Applications

1. Battery-Powered IoT Devices
2. Used in smart devices to optimize power consumption and extend battery life.
3. Smart Home Systems
4. Helps in energy-efficient operation of sensors, lights, and automation systems.
5. Wearable Devices
6. Ensures low power usage in fitness bands and smartwatches.
7. Industrial Automation
8. Monitors and reduces power consumption in real-time control systems.
9. Electric Vehicles (EVs)
10. Useful for monitoring battery usage and improving energy efficiency.
11. Wireless Sensor Networks
12. Enhances lifetime of sensor nodes by using sleep and idle modes.
13. Medical Devices
14. Applied in portable medical equipment where power efficiency is critical.
15. Environmental Monitoring Systems
16. Used in remote sensing systems that rely on limited power sources

OUTPUT



```
===== ACTIVE MODE =====
LED: ON | Sensor: ON | CPU: Full speed
[0s] ACTIVE          | V=4.612V I=24.20mA P=108.000mW
-----
[0s] Distance: OUT OF RANGE
[1s] ACTIVE          | V=4.732V I=2.40mA P=10.000mW
-----
[1s] Distance: 236.24 cm
[2s] ACTIVE          | V=4.736V I=2.50mA P=14.000mW
-----
[2s] Distance: 270.54 cm
[3s] ACTIVE          | V=4.728V I=2.30mA P=10.000mW
-----
[3s] Distance: 236.26 cm
[4s] ACTIVE          | V=4.728V I=2.50mA P=14.000mW
-----
[4s] Distance: 236.24 cm
```

```
===== IDLE MODE =====
LED: OFF | Sensor: OFF | CPU: Running (low work)
[5s] IDLE            | V=4.736V I=2.70mA P=14.000mW
-----
[6s] IDLE            | V=4.744V I=2.60mA P=14.000mW
-----
[7s] IDLE            | V=4.744V I=2.50mA P=14.000mW
-----
[8s] IDLE            | V=4.744V I=2.70mA P=14.000mW
-----
[9s] IDLE            | V=4.744V I=2.20mA P=10.000mW
-----

===== LIGHT SLEEP =====
CPU: Paused | RAM: Retained | Wakeup: 3s timer
[10s] BEFORE LIGHT SLEEP | V=4.744V I=2.30mA P=10.000mW
-----

Woke from LIGHT SLEEP
[13s] AFTER LIGHT SLEEP  | V=8.096V I=0.10mA P=0.810mW
-----
[15s] AFTER LIGHT SLEEP  | V=8.196V I=0.10mA P=0.820mW
-----
[16s] AFTER LIGHT SLEEP  | V=8.196V I=0.10mA P=0.820mW
-----
```

```
===== DEEP SLEEP =====
ALL peripherals OFF | RTC only
[17s] BEFORE DEEP SLEEP | V=8.096V I=0.10mA P=0.810mW
-----

->> Entering DEEP SLEEP for 10 seconds <<<
ts Jul 29 2019 12:21:46

:st:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
:conf:0, SPIWP:0xee
:clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
:load:0x3fff0030,len:4640
:load:0x40078000,len:15620
:0 0 tail 12 room 4
:load:0x40080400,len:3164
:entry 0x4008059c
: (64) esp_core_dump_0's dump data check failed:
'calculated checksum='170b4d5b'
'image checksum='ffffff'
```

CONCLUSION

This project focused on the analysis of power dissipation in an embedded system using FreeRTOS on the ESP32 microcontroller. The system was designed to monitor real-time power consumption using the INA219 sensor while executing multiple tasks such as distance measurement and LED indication.

By implementing different operating modes such as Active, Idle, and Sleep, it was observed that power consumption varies significantly depending on task execution and CPU utilization. The results clearly demonstrate that using RTOS-based task scheduling helps in optimizing resource usage and reducing unnecessary power consumption.

The project successfully highlights the importance of efficient power management in modern embedded and IoT systems. It also proves that integrating RTOS with power monitoring sensors can greatly improve system performance and energy efficiency.

Overall, this system provides a practical approach for designing low-power embedded applications and can be extended further for advanced real-time energy optimization solutions.

REFERENCES

FreeRTOS (2024): FreeRTOS Kernel Documentation.

Available at: <https://www.freertos.org>

(Used for implementing real-time task scheduling and power management).

Espressif Systems (2023): ESP32-WROOM-32 Datasheet.

Available at: <https://www.espressif.com>

(Official technical reference for ESP32 microcontroller used in the project).

Arduino (2024): Arduino IDE Documentation.

Available at: <https://www.arduino.cc>

(Development environment used for coding and uploading programs).

IEEE (2023): Research Papers on Power Optimization in Embedded Systems.

Available at: <https://ieeexplore.ieee.org>